



// 000 TECHNICAL WHITEPAPER · V3

The runtime boundary for the AI-native stack.

Secure microVM infrastructure for workloads that execute generated code, call tools on behalf of a model, and process input you didn't write. Isolated. Signed. Policy-governed. Audited. Sub-150 ms boot.

// AUDIENCE

Security, platform &
engineering leaders

// READING TIME

~25 minutes

// REVISION

V3 · 2026.05

What's in here.

A walking tour through how MVM thinks about AI-native execution — the boundary, the contract, the trust zones, the fleet, and what it means in production.

ES	Executive summary	03
	Why the application process is no longer the security boundary, and what MVM does about it.	
01	The boundary moved	04
	What changes when input becomes instruction.	
02	Two systems, one contract	06
	MVM the runtime, mvmd the orchestrator.	
03	Decorated execution plans	08
	The signed document a workload is launched from.	
04	Three trust zones	10
	Where the controls live, and where tenant code does not.	
05	Identity, keys, attestation	12
	Short-lived authority for runs that interpret untrusted input.	
06	Releases, hosts, and the fleet	14
	How mvmd runs the runtime as infrastructure.	
07	Policy, data, artifacts, audit	17
	The governed path between the workload and the world.	
08	Where MVM fits	19
	Operating modes, comparisons, the decision point.	
09	Threat to control	21
	A summary of what each AI-native risk maps to.	
10	Closing	22
	The runtime as the final line of defense.	

// ES EXECUTIVE SUMMARY

Treat execution as a security boundary, not an application feature.

The systems your teams are shipping now don't just respond to requests. They interpret intent, run code a model just generated, call tools on a customer's behalf, process documents they have never seen before, and forward outputs to other systems. Each of those is a control-flow decision made under the influence of input the application did not write.

Classical web infrastructure assumes the application is the trust boundary. Authentication at the edge, validation at the API, and trust in the deployed code to behave the way its tests said it would. AI-native infrastructure has to assume that the application is no longer enough. Behavior is now partly determined by content — a retrieved document, a tool result, a filename, an email body — that the model interprets as instruction the moment it lands in context.

// THE THESIS

A guardrail living in the same address space as the workload it is meant to constrain is not a boundary. It is a hint.

// SECURITY WITHOUT THE LATENCY TAX

The reflex is that hardened isolation makes a system slower. MVM is built so it doesn't. MicroVMs cold-boot in under 150 ms. The supervisor sits beside the workload, not across a network — egress mediation is in-process, not a hop. Sleep preserves tenant state, so warm wake is immediate. The boundary moves; the response time doesn't.

MVM is secure microVM infrastructure for running AI-native workloads. Each workload runs inside an isolated, signed, policy-governed environment. Outbound activity is mediated by infrastructure the workload cannot reach. Credentials are short-lived and scoped to the run. Releases, hosts, keys, and policy are all governed objects with audit and rollback.

A control plane called `mvmd` runs the fleet on top of all of this. It places workloads on the right hosts, manages sleep and wake, drives releases, gates key release on attestation, and handles recovery — across local development, hosted infrastructure, and confidential-computing hardware.

The result is an execution substrate where you can give an agent meaningful capability without giving it ambient authority — where a compromised workload still cannot exfiltrate, escalate, or move laterally, and where "what ran, under what authority, on what host" is something a security team can answer from infrastructure events the workload could not corrupt.

Concretely: a microVM runtime plus an orchestrator (`mvmd`) that govern tenants, releases, keys, policy, hosts, and audit across the fleet. One contract — signed image, decorated execution plan, scoped credentials, audited lifecycle — runs identically on Firecracker, Lima, Incus, containerd, and adapter-driven backends; on laptops, hosted infrastructure, edge, and confidential-computing hardware (AMD SEV-SNP, Intel TDX) where available.

// 01 THE PROBLEM

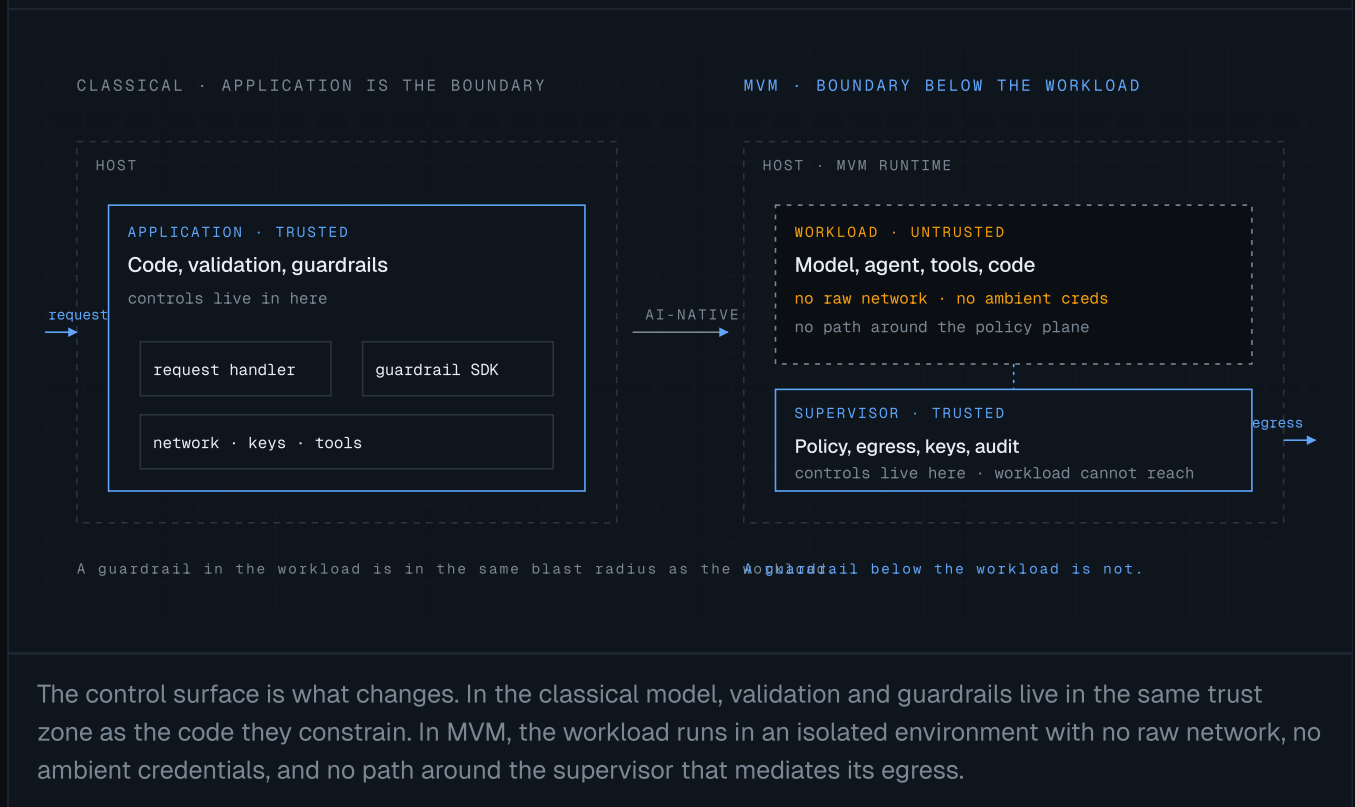
The boundary moved.

For two decades, the trust boundary in a web application has lived at the application process. Authentication at the edge. Validation at the API layer. The code we deploy is trusted to do approximately what its tests said it would, because the code's behavior is determined by the code itself.

AI-native systems break that assumption. Behavior is now partly determined by input — a document, a tool result, an email body, a filename, a log line — that the model interprets as instruction the moment it lands in context. A retrieval pipeline can pull a poisoned page. A user can upload a PDF that quietly redirects an agent. A tool result can carry next-step guidance the developer never wrote. The application process has become an intermediary between an untrusted instruction stream and the rest of your environment.

FIG.01 TRUST BOUNDARY · BEFORE / AFTER

SCALE 1:1



The control surface is what changes. In the classical model, validation and guardrails live in the same trust zone as the code they constrain. In MVM, the workload runs in an isolated environment with no raw network, no ambient credentials, and no path around the supervisor that mediates its egress.

The threat surface, in plain terms

Modern AI applications don't have one input channel — they have many, and most can carry instruction. Prompt injection is the headline: ordinary content becomes runtime control flow once a model treats it as instruction.

Tools are the second surface. The same capabilities you intended for a benign workload — reading files, calling APIs, sending messages — become lateral movement when a workload is manipulated. Dynamic runtimes are the third: Python is powerful precisely because it can load modules, spawn processes, and reach the network, which makes it a poor neighbor in a multi-tenant process. And model output itself is an egress channel that can compose its own exfiltration payload.

Underneath all of that, the supply chain has expanded. Prompts, policy bundles, model configs, retrieval indices, fine-tuned weights, runtime images, Python environments — every one is an artifact that needs provenance.

The central infrastructure question is not whether the model can be made perfectly safe. It is whether the runtime constrains what any model-mediated workload is allowed to access, execute, emit, and persist.

// THE PRACTICAL CONSEQUENCE

Any control inside the workload's address space is, by construction, in the same blast radius as the workload itself. A prompt-injected workload with the same credentials as a benign one can defeat every check that lives in its own process. The controls have to move.

MVM moves them. Egress mediation, PII inspection, exploit detection, secret release, tool-call validation, audit emission — all of it lives outside the tenant workload, in a runtime path the workload cannot reach. That is the property the rest of this document is about.

What this means for your existing stack

MVM is not a replacement for your application code, your model layer, or your guardrail tooling. Those still do useful work — they're just not the security boundary anymore. MVM is the substrate underneath them: the thing that makes "what the workload was allowed to do" a property of infrastructure, not of trust in the workload.

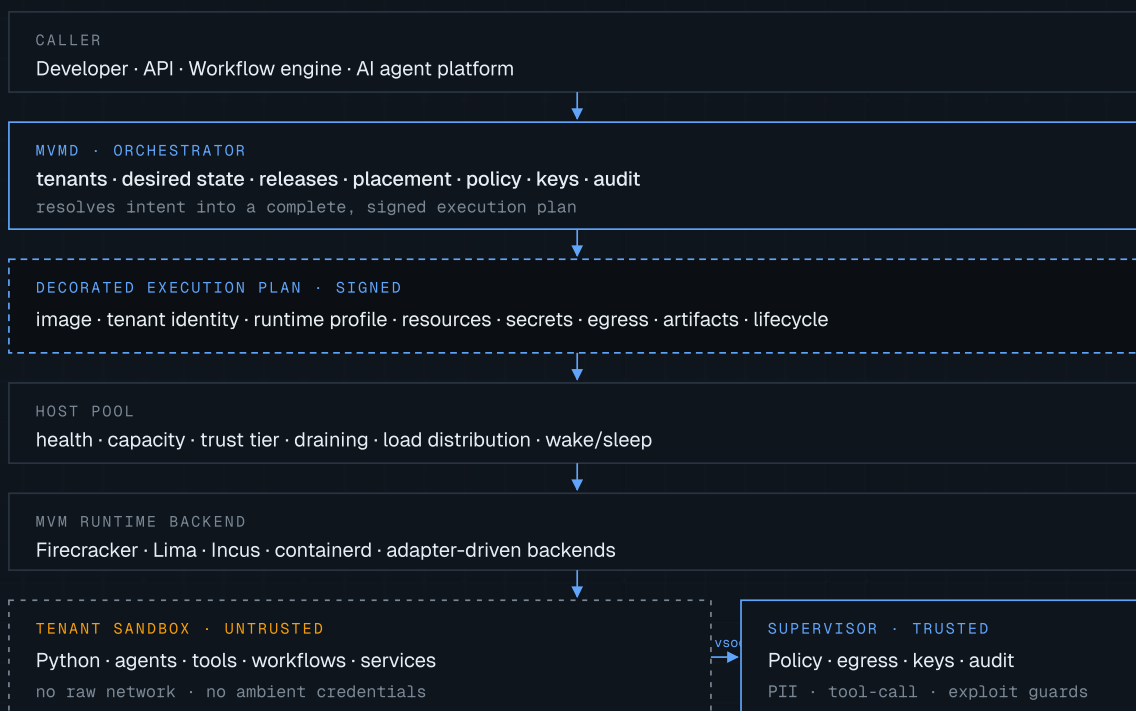
Two systems, one contract.

MVM is two things working together. The runtime executes a single workload on a single host — verifies the image, constructs the sandbox, mediates the egress path, captures audit, and tears the environment back down. mvmd turns that one-off execution into governed infrastructure across a fleet.

The runtime speaks several execution backends. Firecracker is the primary backend, well suited to high-density, short-lived microVM workloads on Linux/KVM, with cold boot under 150 ms — fast enough that isolation per request, per tool call, or per agent step is operationally cheap. Lima covers macOS and developer machines. Incus integrates with VM/container infrastructure. containerd plugs into the OCI ecosystem. Additional hypervisors and microVMs land through backend adapters. The choice of backend is a deployment detail. The contract — signed image, decorated execution plan, isolated tenant, mediated egress, scoped credentials, audit trail — is the same on every one.

FIG.02 REFERENCE ARCHITECTURE · INTENT TO AUDIT

SCALE 1:1



Intent enters at the top, gets resolved into a signed and decorated execution plan, gets placed on a host, and runs in a tenant sandbox whose only path to the world is through the supervisor. Every horizontal line in this diagram is a governed transition with audit.

mvmd is a control system, not a launcher

A launcher takes a command and starts a process. A control system owns the difference between a workload that *exists* and one that has been admitted, signed, placed, credentialed, observed, and audited. mvmd does the second job, continuously.

On the way in, it admits a workload only when a host's capabilities and trust tier match the plan, distributes the signed policy bundle, and releases credentials scoped to the run. While the workload is alive, it watches the fleet, coordinates rollouts and rollbacks, and drains hosts for maintenance. Underneath all of it, every transition is recorded — with enough fidelity that an incident review months later can still answer one question: *what authority did this workload have, and why.*

The runtime is also more than a process boundary. It owns the trust zones described in section 04, the policy plane described in section 07, the key custody described in section 05, and the lifecycle described in section 06. What the workload sees is its sandbox. What everyone else sees is governed infrastructure on either side of it.

The contract is what survives backend choice

Backend abstraction is sometimes treated as a feature for portability. Here it's a property of the threat model. Production work runs on KVM-based microVMs. Developer work runs on macOS or local Linux. Edge work runs where data must stay local. Confidential work runs on SEV-SNP or TDX. The same workload identity, signing, policy, audit, key, and lifecycle concepts apply on every one. Where a backend offers different isolation or attestation properties, mvmd records the trust tier and lets policy decide whether sensitive material may be released.

```
// PUT ANOTHER WAY
```

The product is not the hypervisor. The product is the governed contract around it. Firecracker is a first-class backend. It is not the boundary.

What lives in MVM, what lives in mvmd

Layer	Lives in	What it does for you
Image & decorated plan	mvmd	Verifiable artifact identity and a signed record of what was admitted, under what policy.
Sandbox isolation	runtime	MicroVM- or VM-grade isolation around tenant code.
Egress / tool / PII path	runtime supervisor	An outbound path the workload literally cannot bypass.
Keys, secrets, attestation	mvmd + supervisor	Short-lived authority, scoped per run, gated on policy and (where available) hardware.
Releases & audit	mvmd	Releases as governed objects. An audit trail that survives compromised tenant code.

// 03 THE CONTRACT

You don't run a workload. You submit an **intent**.

In MVM, the thing that gets executed is not a command. It is a *decorated execution plan*: a signed document that captures everything a host needs to know to run the workload safely.

mvmd builds the plan from caller intent. It attaches tenant scope and runtime profile, resolves secret references, pins the policy version that will govern the run, declares the keys the workload may receive, and writes the lifecycle behavior that will follow. The plan is then signed before it leaves the control plane.

The practical effect is that workloads are not launched from ambiguous user requests. They are launched from infrastructure-grade documents that any auditor or operator can read. If something goes wrong six weeks from now, the answer to "what ran, under what policy, with what credentials, on what host" is the plan itself.

FIG.03 DECORATED EXECUTION PLAN · ANATOMY

SCALE 1:1

CALLER INTENT

"run agent X for"
"tenant Y, with"
"policy Z"



SIGNED · DECORATED EXECUTION PLAN

v1.4.2 · sha256:7c...

// IDENTITY

tenant · workload · runtime profile · image ref & signature

// RESOURCES

CPU · memory · disk · process limits · runtime timeout

// ACCESS

network policy · filesystem & volume policy · secret references

// GOVERNANCE

PII / egress policy · tool-call policy · attestation tier · policy version pin

// LIFECYCLE

artifact retention · sleep / wake · key rotation · post-run behavior

// AUDIT

labels · cost & usage metadata · release pinning · signature chain

Caller intent is small and ambiguous. The decorated execution plan is large, complete, and signed. Everything needed to admit, place, credential, audit, and reason about the workload travels with it.

Why "decoration" is not just metadata

Decoration is the step that converts a developer-shaped request into a fleet-shaped artifact. A developer asks to run a workload; mvmd attaches the tenant identity it must run as, the runtime profile it is allowed to consume, the secret references it is permitted to dereference, the egress policy that will mediate it, the trust tier required to admit it, and the lifecycle that will follow it. The plan is then signed. From that moment until the run completes and audit is sealed, the plan is the source of truth.

This is what makes silent runtime mutation impossible. A workload can't quietly be granted broader network access, a higher trust tier, a longer-lived credential, or a different policy version mid-run — the change would require a new signed plan, which is itself an audited operation. What admitted the workload and what governs it are the same document.

What this gives a security team

// WITH DECORATED PLANS

"What ran" and "under what authority" are the same answer. Audit is forensic. Rollback is precise. Compliance reviews can read documents, not reverse-engineer behavior from logs.

// WITHOUT

"What ran" lives in deploy logs. "Under what authority" lives in environment variables, secret managers, and tribal knowledge. Reconstruction is archaeology.

What it gives a platform team

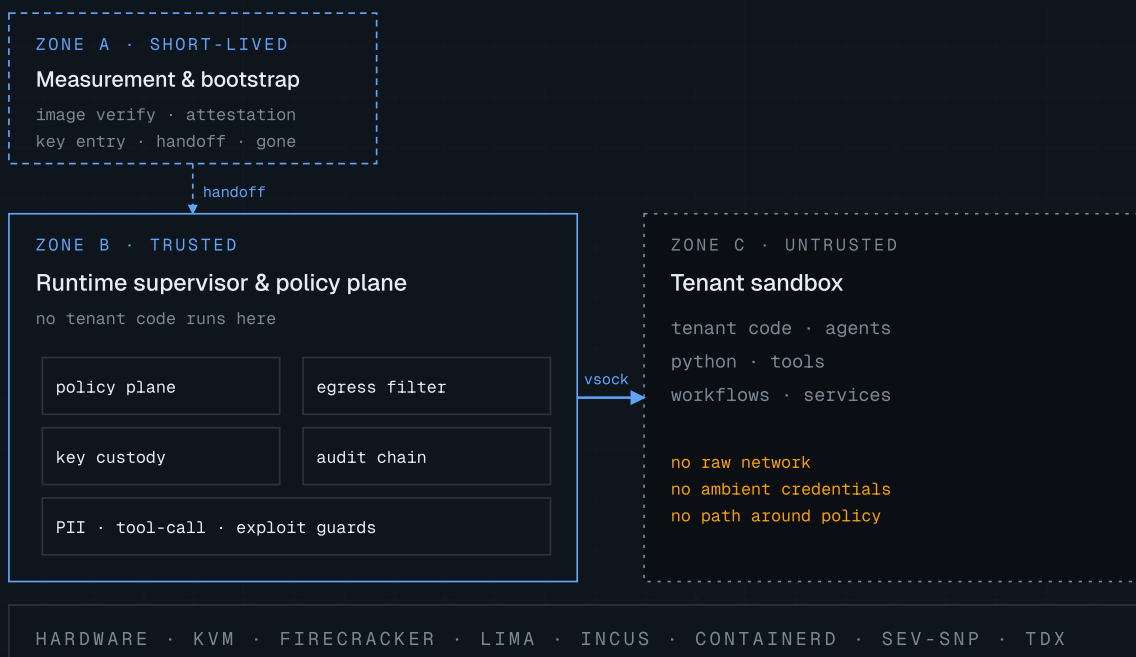
Decorated plans are also where workload portability becomes real. The same plan, with the same signature, runs identically on Firecracker in production, Lima on a developer's MacBook, and a confidential-computing host when the trust tier requires it. The runtime backend changes; the contract does not. Operators can promote, hold, roll back, drain, and migrate at the level of *the plan*, not at the level of *this random instance on host 17*.

Three zones, three trust levels.

Every running workload is divided into three zones with different trust levels. The split is not a metaphor — they are physically separate, with different responsibilities and different blast radii. The exact process layout varies by backend; the trust model does not.

FIG.04 TRUST ZONES · A · B · C · HW

SCALE 1:1



Zone A measures, attests, and exits. Zone B is the trusted supervisor — policy, egress, keys, audit. Zone C is the tenant sandbox, treated as untrusted. The only path from C to the world goes through B.

Zone A · Measurement, bootstrap, and key entry

Small, deterministic, short-lived. Zone A verifies the image, participates in attestation if the platform supports it, exchanges initial keys, unseals secrets into the right scope, and hands off to the runtime supervisor. Once its job is done, it is gone. The reason for the split is blast radius: the code that handles initial trust should be small enough to reason about and brief enough that it doesn't outlast the trust it established.

Zone B · The runtime supervisor

Trusted infrastructure. Zone B launches and watches the tenant sandbox, maintains the policy-enforced egress path, applies PII and exploit controls, holds short-lived credentials, coordinates artifact capture, and emits the audit chain. No tenant code ever runs here. This is the part of the runtime that makes "the workload was compromised" different from "the system was compromised."

Zone C · The tenant sandbox

Untrusted by design. Customer code, agent code, Python runtimes, workflow nodes, tools, application processes — all of it runs here, with no raw socket to the network, no long-lived credentials, and no path around the policy plane. Zone C speaks to Zone B over an explicit guest-host channel — typically vsock — that the surrounding runtime defines and constrains.

```
// WHY THIS DIFFERS FROM A GUARDRAIL LIBRARY
```

A library lives in Zone C, where it can be omitted, downgraded, misconfigured, or simply walked around by a workload that knows how to open its own socket. MVM's controls live in Zone B, where the workload literally cannot reach them.

Control plane and data plane stay separate, too

The same separation applies above the runtime. The control plane carries desired state, lifecycle commands, host health, policy versions, key rotation events, release transitions, and audit. The sandbox data plane carries the workload's mediated egress, tool calls, and artifact movement. They share transport patterns where convenient, but they are governed independently and audited separately.

A tenant workload cannot mutate control-plane state through its data plane. A compromised tenant cannot promote a release, drain a host, or rotate a key. This is what keeps "compromised workload" from escalating into "compromised fleet."

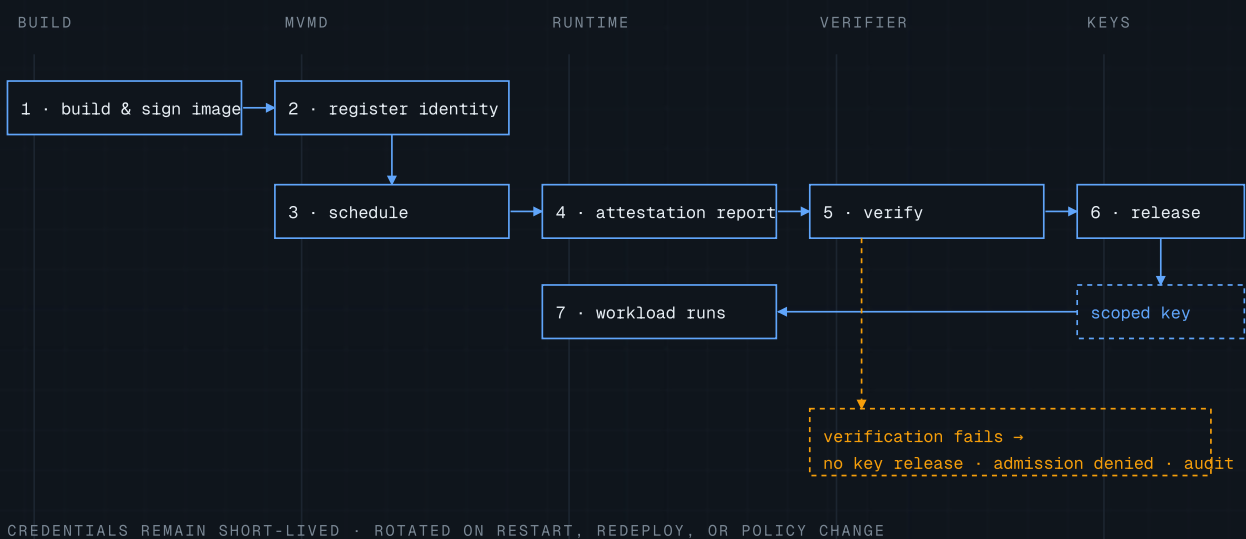
Authority that expires with the run.

Credentials in AI-native systems are unusually pressured. A single agent might need an API key for a model provider, a database token, a webhook secret, a storage credential, and a short-lived grant to call an internal API. If any of those become ambient, long-lived capabilities inside the workload, then a compromised run becomes an indefinite breach.

MVM treats credentials as something the runtime hands out for a specific run, scoped to a specific plan, and revokes when the run ends. Workload secrets are released only into the execution context that has been authorized to use them, gated on policy and (where the hardware supports it) attestation. They expire on workload restart, on policy change, on quarantine, on lifecycle transition. The audit record captures which workload requested access, which tenant owned it, which policy version authorized it, which key version was used, when rotation occurred, and which host or runtime identity participated. *What ran* and *what authority it had while running* are the same question, with the same answer.

FIG.05 ATTESTATION & KEY-RELEASE FLOW

SCALE 1:1



Where confidential-computing hardware is available, attestation gates key release. Without it, the same workload contract still runs — the policy tier records what guarantees were possible, and refuses to release sensitive material into an environment that cannot carry them.

Confidential computing where it matters

MVM uses hardware-rooted confidentiality where the platform supports it — AMD SEV-SNP and Intel TDX in particular. On a compatible host, a workload's image measurement is verified, its attestation report is checked for measurement, platform identity, freshness, and policy, and only then are sensitive keys released. The same workload contract runs on hardware without these features; the difference is recorded as a trust tier on every release, and the policy decides whether that tier is sufficient for the data class involved.

The point is not "confidential computing for everything." It is "the runtime knows what guarantees it can make, and refuses to make ones it can't." A workload that handles regulated data can require a tier the available host pool cannot satisfy, in which case it is not admitted. That refusal is itself an audited event.

Protecting the control plane itself

The same pattern protects mvmd. Its authority is split into role-scoped, short-lived, rotated, revocable credentials. A compromised control-plane key does not grant unlimited, silent authority — it produces audit, it is bounded by signing and policy versioning, and it can be revoked, the policy rolled back, affected workloads quarantined, and affected hosts drained. The control plane assumes its own potential compromise, and is built to contain the blast radius of one.

// THE GUARANTEE

A compromised credential — workload or control plane — should produce a recoverable, auditable, and bounded incident. Not an indefinite, silent, ambient breach. That distinction is the difference between an infrastructure trust model and a wishful one.

What audit captures, in plain language

For every key event: which workload requested access, which tenant owned it, which policy version authorized it, which key version was used, when rotation occurred, whether revocation succeeded, and which host or runtime identity participated. Security teams can answer not only what ran, but what authority it had while running, and what changed afterward to take that authority away.

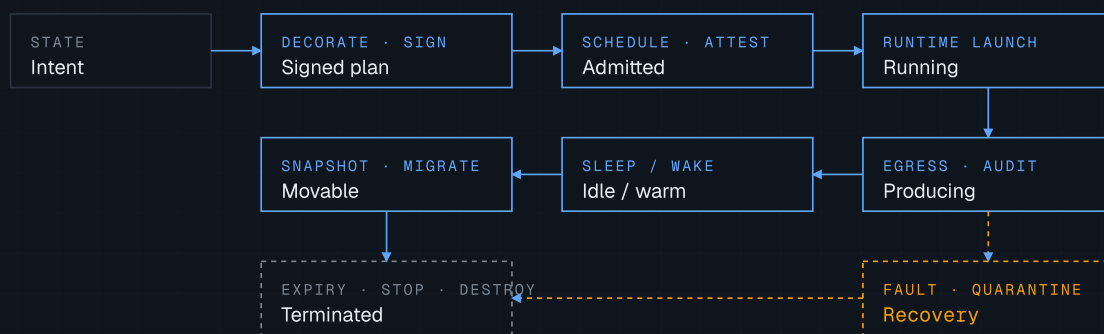
Releases as governed objects.

A release in MVM is not just a new image. It is a coordinated change — to the image, to the execution plan, to the policy bundle, to the keys the workload may receive, to how its artifacts are handled. Each piece has security consequences, so mvmd treats releases the way a serious infrastructure team treats production change: signed artifacts, pinned versions, staged rollout, canary, controlled rollback. A workload upgrade is never a silent host-side operation.

Hosts are first-class participants. A host is not just a machine that *can* run a workload — it is an identified participant with a known trust tier, a tracked health and capacity, and an audit history of every workload it has accepted. mvmd watches its health, places workloads onto it within its envelope, drains it for maintenance, migrates workloads off it during failure, and revokes it when it leaves the fleet. Every transition is a logged event.

FIG.06 WORKLOAD LIFECYCLE · STATE TRANSITIONS

SCALE 1:1



EVERY EDGE IS AN AUDITED EVENT · MVMD CONTINUOUSLY RECONCILES OBSERVED STATE AGAINST DESIRED STATE

Workloads enter as intent and exit through governed terminal states. Failures branch into a recovery path that mvmd drives back toward a known-good state.

Sleep and wake aren't just a cost feature

AI-native traffic is bursty. Interactive agents need warm capacity. Background jobs run irregularly. Demand spikes are common. So the fleet sleeps and wakes on its own — idle workloads release their capacity, tenant state is preserved, and demand wakes them on the next request. Warm pools absorb interactive bursts. Snapshots make wake transitions cheap where the backend supports them: a sleeping workload returns in tens of milliseconds rather than seconds, so isolation per request stays inside an interactive budget.

The reason this is in the security model and not a footnote is that the same machinery that places a release also places a wake. If wake/sleep ran outside the governed path, then idle-capacity tricks would be a way to bypass policy — start a workload at a less-watched moment, accumulate authority, and outrun the audit window. In MVM, every sleep, wake, snapshot, and migrate is a logged transition, on the same plane that placed the workload originally.

Failure as part of the control model

Recovery is not a separate runbook. mvmd continuously compares desired state against observed state, and drives the system back toward governed. Every failure has a recovery behavior, and every recovery behavior is itself audited.

Failure	Recovery behavior
Workload crash	Preserve audit context, collect artifacts, apply retry or lifecycle policy.
Host crash	Mark host unhealthy, reconcile affected workloads, migrate or restart elsewhere.
mvmd restart	Rehydrate desired state, reconcile against observed state, resume.
Release failure	Stop rollout, preserve audit, roll back or hold affected workloads.
Key rotation failure	Deny new sensitive operations, retry rotation, revoke affected grants.
Policy distribution failure	Pin to known versions, or deny admission when freshness is required.
Failed attestation	Deny key release, refuse admission to attestation-required tier.
Stuck sleep / wake	Reconcile runtime state, retry, migrate, or escalate for operator review.
Overloaded host	Stop placing new workloads, migrate eligible work, wake additional capacity.

Hosts as governed participants, not anonymous workers

Treating hosts as identified, audited participants rather than anonymous workers is what lets mvmd enforce data residency, attestation tiers, and trust boundaries at placement time. A workload that requires confidential computing is not placed on a host that cannot provide it. A workload bound to a residency region is not placed outside it. A host that has lost eligibility is not given new work. None of this requires the workload to ask the right question; it is enforced by the placer, audited as a release event, and visible after the fact.

Operationally: what a workload upgrade looks like

// IN MVM

Sign new image. Update plan. Promote release through canary. Drain old hosts gracefully. Migrate workloads. Roll back atomically if anything trips a guard. Every step audited.

// WITHOUT

SSH to host. Pull new container. Restart service. Hope nothing on it was state-bearing. Discover the regression in production. Reconstruct what changed from chat logs.

The first version is governed infrastructure. The second is operations as folklore. The infrastructure cost of the first is real; the recovery cost of the second is unbounded.

Policy is a first-class runtime artifact.

Policy in MVM is not a library setting. It is a signed, versioned artifact, distributed through the same governed path as workload images. Each bundle answers a clear set of questions for the workload it governs: where it may reach, what tools it may call, which PII rules apply, which exploit controls run, which keys it may receive, what becomes of its artifacts, and what gets audited.

Decorated execution plans pin the exact policy version that admitted the workload. Behavior is explainable months later, and silent runtime mutation is impossible.

When something becomes urgently unsafe — a destination, a tool, a model, a webhook, a package source — emergency deny rules ship as signed policy updates through the same path. Fast does not mean ungoverned.

Egress is where most of policy meets the network

Tenant workloads do not get an unconstrained outbound socket. They get a mediated path that does several jobs at once: it enforces destination allow/deny lists and normalizes protocols, validates tool calls, and inspects payloads for PII, secrets, SSRF attempts, and injection patterns. Volume anomalies get flagged. Every decision — allowed, transformed, or denied — is audited.

Because this enforcement is in Zone B, tenant code cannot skip it, no matter how compromised. There is no socket the workload can open that bypasses it — because that socket does not exist inside the workload's environment.

// COMPARE: SDK GUARDRAILS

An SDK-level guardrail lives inside the application's trust boundary. It can be omitted, downgraded, misconfigured, or bypassed by compromised code. MVM's egress path lives outside the tenant workload. The boundary is in the runtime, not in your application code.

Data classification, residency, and placement

Different workloads handle different classes of data, and policy reflects that. A workload processing public documentation is governed differently from one processing customer PII or regulated data. Data classification feeds directly into placement: it decides where the workload may run, what trust tier it requires, how its artifacts are retained, and which secrets are released to it.

The practical effect is that "sensitive workload ran in the wrong place" becomes something the system *refuses to do*, not something you discover later. The same model carries across local, edge, hosted, and confidential-computing environments. A workload can redact PII locally before sending a transformed request to a hosted model. A developer can test the same image locally before promoting it to production. A hosted environment can require confidential-computing support before secrets are released. The data path is unified; what differs is the trust tier each environment can offer.

Artifacts as governed outputs

AI-native workloads produce a lot of output — generated files, transformed datasets, model responses, screenshots, traces, embeddings, workflow results. These are not incidental files left behind by a worker; they are outputs of a controlled execution environment.

So policy governs them the way it governs inputs: who owns them, how long they live, who may read them, whether they are encrypted or signed, when they get cleaned up, and what audit records follow them. Generated artifacts can carry sensitive data, derived customer information, or intermediate model outputs. They deserve the same seriousness.

Multi-tenant isolation, in the runtime not the application

A tenant might be a customer, an organization, a team, or just an agent execution scope. Whatever it is, MVM isolates it across every dimension that matters.

Compute runs in separate runtime environments, not shared application processes, with per-workload resource limits. Network gives each tenant its own scope; cross-tenant traffic is denied unless explicitly modeled. Storage is segmented by class — runtime state, workload volumes, secrets, snapshots, artifacts, and audit each have their own retention, encryption, and cleanup rules. Secrets are never shared across tenants and never live as static files inside workload images. Audit records are scoped per tenant and per workload.

Tenant isolation in the application layer depends on the application getting it right on every request. Tenant isolation in the runtime depends on the runtime getting it right once.

Audit, where it matters most

The audit trail is the thread through all of this. For every workload it answers a small, precise set of questions: who submitted it, which tenant owned it, which image and release ran, which policy applied, which host accepted it, which keys it received, which egress decisions were made, which artifacts it produced, and how its lifecycle ended.

Crucially, the audit lives in the control plane — not in the workload's own logs. So it remains usable even when the workload is compromised, buggy, or has nothing to say. *What happened* and *under what authority* are questions application logs alone can rarely answer; this trail can.

// PRACTICAL OUTCOME

Incident response, compliance review, and SOC pivots all become document operations against a signed event stream — not archaeology against application logs that the workload itself could have corrupted.

Where MVM fits.

MVM is a strong fit wherever a workload is powerful enough to be useful: AI agent runtimes that interpret model output as control flow, Python interpreters running generated code, workflow engines executing tool steps, multi-tenant build systems, hosted developer environments, edge deployments where data must stay local, internal automation platforms that touch credentials and customer data. The need is sharpest when workloads can reach sensitive files, credentials, user data, private APIs, or model-mediated tools — which is most of the systems being built right now.

Six operating modes, one contract

// MODE.01

Local Development

The same signed workload and policy model on a developer's machine. Lima or compatible backend. Iterate locally, ship the same artifact to production.

// MODE.02

Hosted Standard

Isolated signed workloads on managed hosts with tenant, policy, key, artifact, and audit controls. Firecracker as primary backend.

// MODE.03

Hosted Confidential

Hardware attestation and confidential-computing support gate sensitive key release. AMD SEV-SNP and Intel TDX as native targets.

// MODE.04

Edge or Private

Run close to sensitive data while preserving signing, policy, lifecycle, and audit semantics. Same execution contract.

// MODE.05

Build & Artifact

Secure builders and artifact-generation workloads inside governed execution environments. Output artifacts inherit policy.

// MODE.06

Workflow Runtime

Workflow nodes, Python steps, tools, and agents run with tenant-scoped policy, key release, and audit.

Operating mode changes placement and trust requirements; it does not change the core execution model. Anything that needs to run governed workloads — workflow engines, AI-agent platforms, secure builders, developer sandboxes, internal automation — submits into the same execution contract through MVM's CLI, API, and SDK hooks.

Compared to what you might already be using

MVM is sometimes evaluated against tools that solve a slice of the problem. Here is how to think about the difference.

Approach	What it does well	Where MVM is different
Containers	Packaging, process isolation, OCI ecosystem.	Stronger workload isolation when the threat model requires it; runtime policy outside the tenant workload, not inside it.
Conventional VMs	Strong isolation, mature operations.	Built specifically for high-density, ephemeral, policy-rich AI workloads — with signing, tenant scope, key management, release safety, and lifecycle automation.
Guardrail libraries	Application-layer hints, fast iteration on rules.	Policy enforcement lives outside the tenant sandbox, where compromised code cannot omit, downgrade, misconfigure, or walk around it.
CI runners	Pipeline execution at scale.	A reusable runtime layer for long-lived agents, tools, workflows, and hosted AI services — not just builds.
Ad hoc sandboxes	One slice solved well — process isolation, network filtering, logging, dependency control.	Treats isolation, orchestration, releases, policy, secrets, images, audit, and artifacts as one system.

The decision point

MVM is not the right answer for every workload. It is the right answer for workloads that are powerful enough to do something you would regret — execute generated code, call tools on a customer's behalf, process unfamiliar input, move data across boundaries. The decision is straightforward.

// THE TEST

If a workload is powerful enough to be useful, it is powerful enough to require a real runtime boundary. MVM provides that boundary as infrastructure.

Threat to **control**, at a glance.

The mapping below is intentionally infrastructure-oriented. MVM does not depend on perfect model behavior. It constrains the runtime consequences of model-mediated behavior — which is what makes it suitable as infrastructure rather than as application logic. Each row connects something a security team worries about to the part of the system that handles it.

Threat	Where MVM addresses it
Prompt injection causes tool abuse	Tool-call policy and mediated egress in Zone B, outside the tenant workload.
Model output leaks PII	PII inspection, redaction, masking, blocking, or quarantine at the egress path.
Tenant attempts cross-tenant access	Tenant-scoped compute, network, storage, secrets, artifacts, and audit.
Host receives tampered image	Signed image verification and content identity checks; unsigned artifacts refused.
Policy changes silently alter behavior	Signed policy bundles, version pinning, rollback, and audit of every change.
Long-lived secret leaks	Short-lived credentials, scoped release, rotation, and revocation as part of the run.
Bad release ships	Staged rollout, canary, host draining, controlled rollback, release audit.
Demand spike or unhealthy host	Load-aware placement, wake-on-demand, health checks, migration, revocation.
Control-plane credential compromised	Role scoping, key rotation, signed desired state, audit, revocation, rollback.
Sensitive workload in wrong location	Data-classification policy, trust-tier requirements, placement denial.
Artifact contains sensitive data	Artifact retention, access control, encryption, expiration, audit.
Attestation fails	Key release denied, admission to attestation-required tier refused.

// 10 CLOSING

The runtime is the final line of defense.

The systems being built now do more than respond to requests. They generate code, call tools, process unfamiliar files, and forward outputs to other systems on the basis of input the application did not write. The runtime that holds those systems is one of the most important parts of the security architecture.

MVM provides that runtime as governed infrastructure. Workloads run from signed artifacts, inside isolated environments, behind policy-enforced egress, with scoped and rotating credentials, on a fleet that mvmd manages from admission to audit. Where confidential-computing hardware is available, attestation gates sensitive key release. Where it isn't, the same contract still runs, with the trust tier honestly recorded.

The application process is no longer the final line of defense. The runtime is. That shift — from trusting the workload to constraining it — is what makes a powerful AI system safe enough to give powerful work to.

// THE OPERATING POSTURE

Run the workload. Isolate the tenant. Enforce the policy. Rotate the keys. Govern the artifacts. Audit the execution. Release safely. Sleep idle capacity. Wake on demand. Manage the fleet.

// NEXT

Request access for an evaluation. We pair customers with a reference deployment that mirrors their workload class — Hosted Standard, Hosted Confidential, Edge, or Workflow Runtime — and run a 30-day governed pilot.

// CONTACT

Reach the MVM team via the request-access form on runmvm.com, or read the live FAQ and trust-zones diagram on the homepage.